Graduate Systems (CSE638)

PA02: Cost of Data Movement in Network I/O

Piyush Kandwal
Roll Number: MT25076

7 February 2026

Contents

1 Introduction **4**

2 Part A: Multithreaded Socket Implementations **4**

2.1 Common Design 4

2.2 A1: Two-Copy Socket Implementation 4

2.2.1 Server-Side Implementation 4

2.2.2 Client-Side Implementation 5

2.2.3 Copy Analysis 5

2.3 A2: One-Copy Socket Implementation 5

2.3.1 Server-Side Implementation 5

2.3.2 Client-Side Implementation 6

2.3.3 Copy Analysis 6

2.4 A3: Zero-Copy Socket Implementation 6

2.4.1 Server-Side Implementation 6

2.4.2 Client-Side Implementation 6

2.4.3 Kernel Behavior 7

3 Part B: Profiling and Measurement Methodology **7**

3.1 Profiling Using perf 7

3.2 Metrics Collected 7

3.3 Experimental Setup 8

4 Part C: Automated Experimentation **8**

4.1 Automated Measurement Bash Script 8

4.2 Plot Generation 9

5 Part D: Plotting and Visualization **9**

5.1 Throughput Analysis 9

5.2 Latency Analysis 10

5.3 Cache Miss Behavior 11

5.4 CPU Cycles per Byte 12

6 Part E: Analysis and Reasoning **12**

6.1 Why does zero-copy not always give the best throughput? 12

6.2 Which cache level shows the most reduction in misses and why? 13

6.3 How does thread count interact with cache contention? 13

6.4 At what message size does one-copy outperform two-copy? 13

6.5 At what message size does zero-copy outperform two-copy? 13

6.6 Unexpected result 13

7 AI Tool Usage Declaration **14**
7.1 Components Where AI Assistance Was Used 14
7.1.1 Code Development and Implementation 14
7.1.2 Experimental Design and Data Collection 14
7.1.3 Plot Generation and Report Writing 15
7.2 Responsibility Statement 15

8 Conclusion **15**

9 GitHub Repository **15**

1 Introduction

Modern high-performance network applications are often limited not by raw network bandwidth, but by the overhead of data movement between user space, kernel space, and the network interface. Traditional socket-based communication involves multiple memory copies, which increase CPU utilization and cache pressure.

This assignment experimentally studies the cost of data movement in network I/O by implementing and comparing three TCP-based communication mechanisms:

- **Two-Copy Communication**: Using `send()` / `recv()`
- **One-Copy Communication**: Using `sendmsg()` with scatter–gather I/O
- **Zero-Copy Communication**: Using `sendmsg()` with `MSG_ZEROCOPY`

The implementations are evaluated using micro-architectural metrics such as CPU cycles, cache misses, throughput, latency, and scalability across varying message sizes and thread counts.

2 Part A: Multithreaded Socket Implementations

2.1 Common Design

All implementations follow a TCP client–server architecture. The server accepts multiple concurrent client connections and creates one thread per client. Each server thread repeatedly sends a fixed-size message to its client.

The message is defined as a structure containing eight dynamically allocated string fields, ensuring that all message data is heap-allocated. Message size and thread count are parameterized at runtime. The client receives data continuously for a fixed duration.

2.2 A1: Two-Copy Socket Implementation

The two-copy baseline implementation uses standard `send()` and `recv()` socket primitives. Since `send()` requires a contiguous buffer, the eight heap-allocated message fields are serialized into a temporary user-space buffer before transmission.

2.2.1 Server-Side Implementation

Listing 1: MT25076_Part_A1_Server.c (Simplified)

```
struct message {
    char *fields[8];
};
```

```
void *client_handler(void *arg) {
    struct message msg;
    char *send_buffer = malloc(MSG_SIZE);

    while (running) {
        serialize_message(&msg, send_buffer);
        send(client_fd, send_buffer, MSG_SIZE, 0);
    }
}
```

2.2.2 Client-Side Implementation

Listing 2: MT25076_Part_A1_Client.c (Simplified)

```
while (running) {
    recv(sockfd, recv_buffer, MSG_SIZE, 0);
}
```

2.2.3 Copy Analysis

This implementation incurs the following copies:

- User-space serialization into a contiguous buffer
- User-space buffer copied into kernel socket buffers

Additional internal kernel copies may occur during protocol processing.

2.3 A2: One-Copy Socket Implementation

The one-copy implementation eliminates the explicit user-space serialization step by using `sendmsg()` with scatter–gather I/O. Each message field is transmitted directly using an `iovec` entry.

2.3.1 Server-Side Implementation

Listing 3: MT25076_Part_A2_Server.c (Simplified)

```
struct iovec iov[8];
for (int i = 0; i < 8; i++) {
    iov[i].iov_base = msg.fields[i];
    iov[i].iov_len  = field_size;
```

```
}

struct msghdr hdr = {0};
hdr.msg_iov = iov;
hdr.msg_iovlen = 8;

sendmsg(client_fd, &hdr, 0);
```

2.3.2 Client-Side Implementation

Listing 4: MT25076_Part_A2_Client.c (Simplified)

```
recv(sockfd, recv_buffer, MSG_SIZE, 0);
```

2.3.3 Copy Analysis

By using scatter–gather I/O, the **user-space serialization copy** (copying separate fields into a single contiguous buffer) is eliminated. The only remaining copy is from the user-space vectors into the kernel socket buffer.

2.4 A3: Zero-Copy Socket Implementation

The zero-copy implementation uses `sendmsg()` with the `MSG_ZEROCOPY` flag. The kernel pins user-space pages and performs DMA directly from application buffers, avoiding data copies in the steady state.

2.4.1 Server-Side Implementation

Listing 5: MT25076_Part_A3_Server.c (Simplified)

```
sendmsg(client_fd, &hdr, MSG_ZEROCOPY);

/* Retrieve completion notification */
recvmsg(client_fd, &err_hdr, MSG_ERRQUEUE);
```

2.4.2 Client-Side Implementation

Listing 6: MT25076_Part_A3_Client.c (Simplified)

```
recv(sockfd, recv_buffer, MSG_SIZE, 0);
```

2.4.3 Kernel Behavior

The kernel pins user pages, performs DMA-based transmission, and later reports completion via the socket error queue. This avoids data copies but introduces page pinning and metadata overhead.

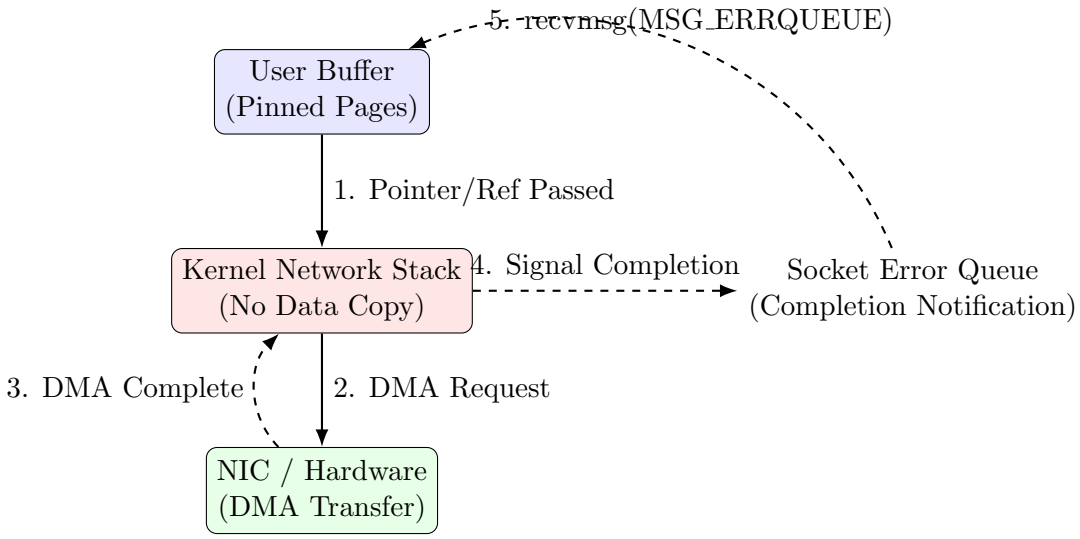


Figure 1: Zero-Copy Data Flow: Data is transferred directly from User Buffer to NIC via DMA. The Kernel manages metadata and signals completion.

3 Part B: Profiling and Measurement Methodology

3.1 Profiling Using perf

Micro-architectural measurements were collected on the server using `perf stat`. The following counters were used:

- CPU cycles
- L1 and LLC cache misses
- Context switches

Listing 7: perf stat Usage (Representative)

```
perf stat -e cycles,cache-misses,context-switches \
    ./MT25076_Part_A1_Server <args>
```

3.2 Metrics Collected

- Throughput (Gbps), Latency (μs) — application-level
- CPU cycles, Cache misses, Context switches — via `perf stat`

3.3 Experimental Setup

All experiments were executed using Linux network namespaces connected via a `veth` pair. Measurements were taken across four message sizes (1K, 4K, 16K, 64K) and four thread counts (1, 2, 4, 8).

4 Part C: Automated Experimentation

4.1 Automated Measurement Bash Script

An automated bash script compiles all implementations, executes experiments across message sizes and thread counts, collects profiling metrics, and stores results in CSV files without manual intervention.

Listing 8: MT25076_Part_C_run_experiments.sh (Simplified)

```
#!/bin/bash
set -euo pipefail

MESSAGE_SIZES=(1024 4096 16384 65536)
THREAD_COUNTS=(1 2 4 8)
DURATION=5
BASE_PORT=5000
IMPLEMENTATIONS=("A1" "A2" "A3")

for impl in "${IMPLEMENTATIONS[@]}"; do
    make clean && make PART=$impl
    for msg in "${MESSAGE_SIZES[@]}"; do
        for th in "${THREAD_COUNTS[@]}"; do
            OUT_FILE="MT25076_Part_C_${impl}_msg${msg}_th${th}.
               csv"
            perf stat -e cycles,cache-misses,context-switches \
                ./MT25076_Part_${impl}_Server $BASE_PORT $msg $th
                    $DURATION &
            SERVER_PID=$!
            sleep 1
            ./MT25076_Part_${impl}_Client 127.0.0.1 $BASE_PORT
               $msg $th $DURATION
            wait $SERVER_PID 2> "$OUT_FILE"
        done
    done
done
```

4.2 Plot Generation

All plots were generated using Python's `matplotlib` library from measured data.

5 Part D: Plotting and Visualization

5.1 Throughput Analysis

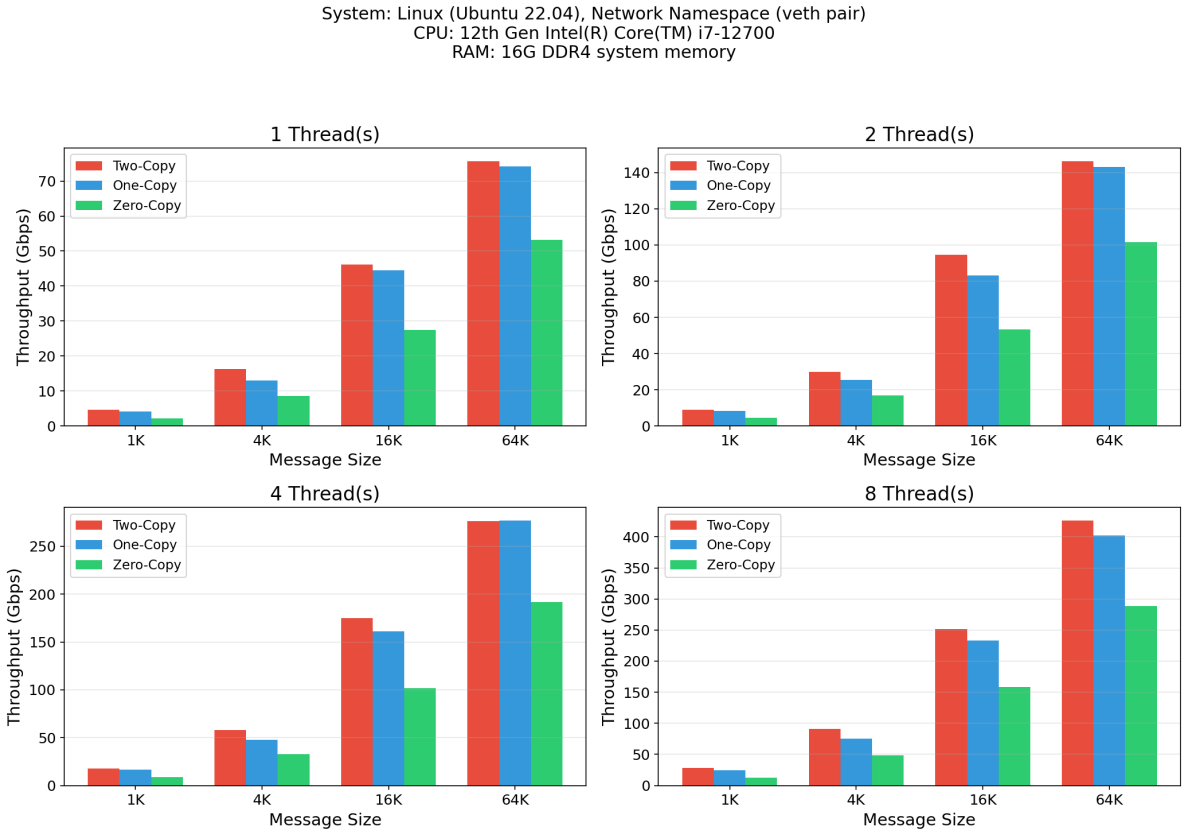


Figure 2: Throughput vs message size. Two-copy consistently achieves the highest throughput, followed by one-copy, with zero-copy trailing across all configurations. Throughput scales with both message size and thread count.

5.2 Latency Analysis

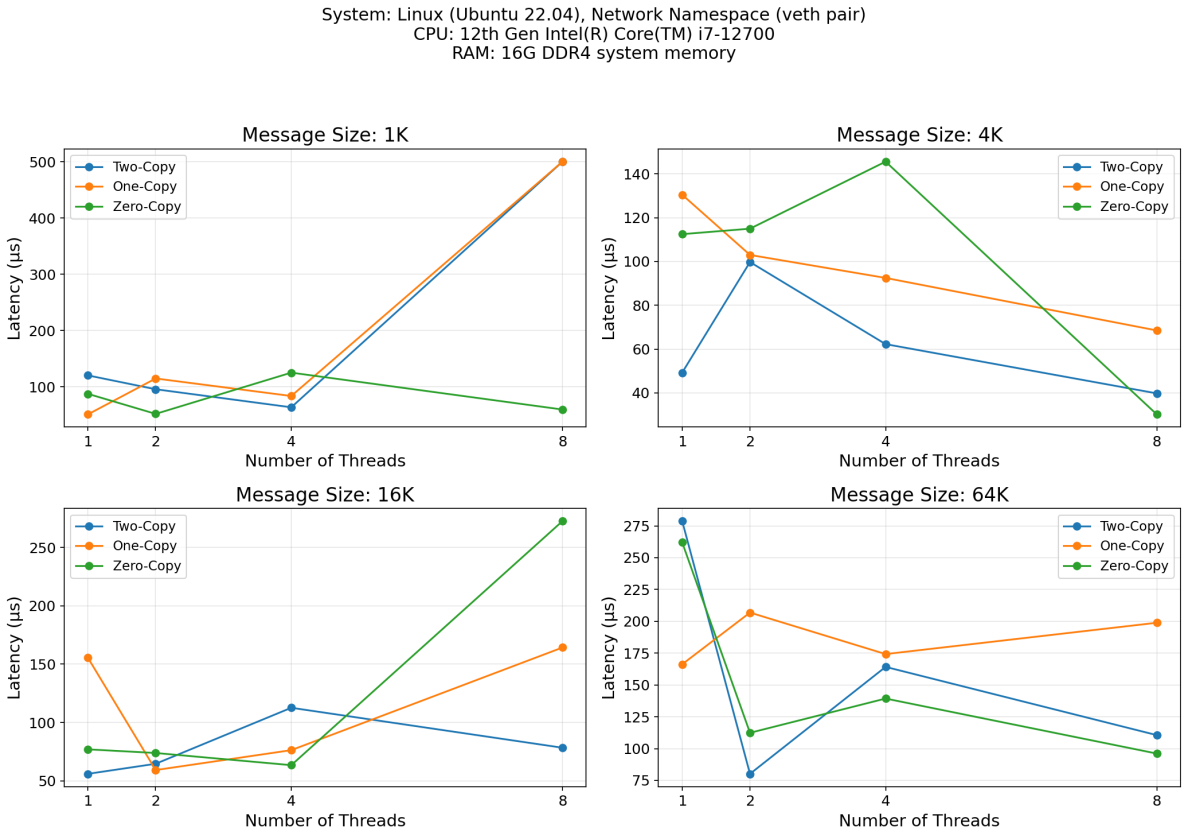


Figure 3: Latency vs thread count. Behavior is non-monotonic: for 1K messages, two-copy latency spikes at 8 threads while zero-copy decreases; for 64K, two-copy latency drops with more threads. These irregular patterns are attributed to per-message latency measurement interacting with parallelism, kernel batching, and P/E-core scheduling on the i7-12700.

5.3 Cache Miss Behavior

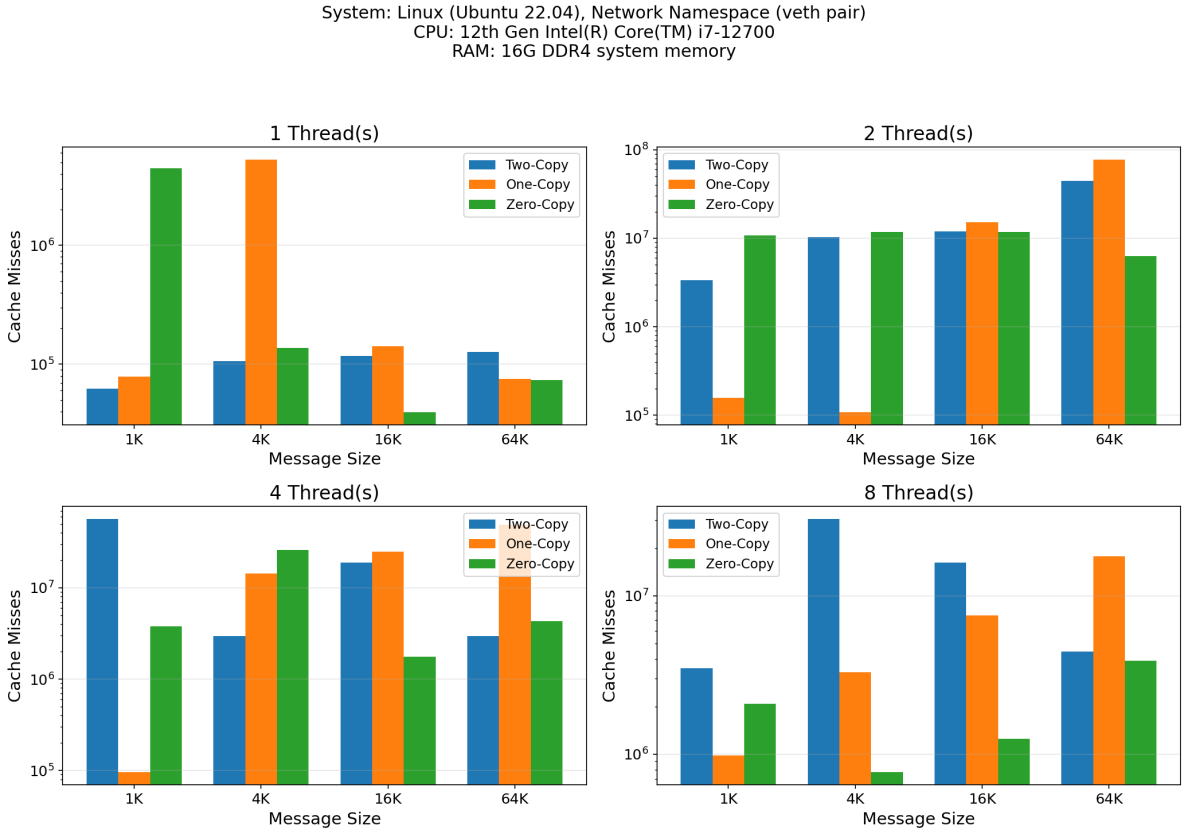


Figure 4: Cache misses vs message size (log scale). No single implementation consistently dominates. Patterns shift across thread counts—e.g., one-copy shows the highest misses at 2 threads, while zero-copy tends toward the lowest misses at 8 threads and large message sizes. The variability is attributed to measurement noise and short experiment duration.

5.4 CPU Cycles per Byte

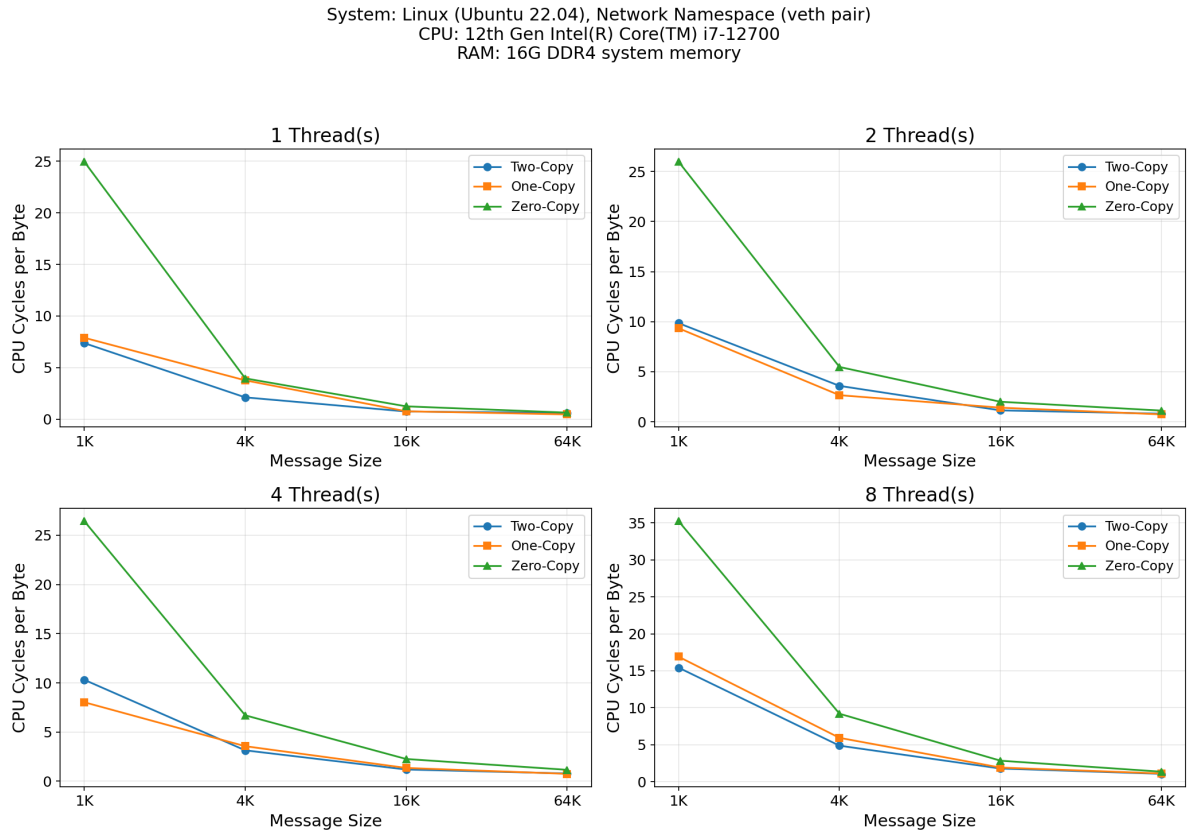


Figure 5: CPU cycles per byte. All implementations show decreasing cost with larger messages. Zero-copy has the highest cycles/byte at small sizes (∼25–35 at 1K) due to page pinning overhead, while two-copy and one-copy remain at ∼7–17. All converge below ∼1 cycle/byte at 64K.

6 Part E: Analysis and Reasoning

6.1 Why does zero-copy not always give the best throughput?

Zero-copy consistently delivers the lowest throughput across all configurations. Its per-send overheads (page pinning, DMA mapping, completion notifications) are not amortized at small/medium message sizes. Additionally, the `veth` pair loops packets within the kernel without real hardware DMA, eliminating zero-copy's primary advantage while retaining all its bookkeeping costs. Modern optimized `memcpy()` makes the actual data copies in two-copy surprisingly cheap.

6.2 Which cache level shows the most reduction in misses and why?

The cache miss data is noisy and does not show a consistent winner. However, at larger message sizes and higher thread counts, zero-copy tends toward lower misses, suggesting LLC benefits from eliminating bulk data streaming. Two-copy and one-copy repeatedly push large buffers through the cache hierarchy, polluting the LLC. L1 behavior is less affected as it is dominated by control structures and metadata rather than bulk data movement.

6.3 How does thread count interact with cache contention?

From the cycles-per-byte plot, zero-copy's overhead at 1K increases from $\sim$25 to $\sim$35 cycles/byte as threads grow from 1 to 8, while two-copy increases from $\sim$7 to $\sim$15. This shows that kernel-level zero-copy operations (page pinning, reference counting) contend more heavily for shared resources at higher concurrency. Cache miss patterns also become more variable at higher thread counts, reflecting LLC contention among concurrent threads.

6.4 At what message size does one-copy outperform two-copy?

One-copy does not clearly outperform two-copy at any tested message size. Two-copy's contiguous buffer layout benefits from hardware prefetching, while one-copy's scatter–gather I/O from non-contiguous addresses introduces TLB overhead that offsets the savings from eliminating serialization.

6.5 At what message size does zero-copy outperform two-copy?

Zero-copy does not outperform two-copy at any evaluated size (1K–64K). The `veth` pair setup means no real DMA benefit is realized. Larger message sizes (e.g., 256K+) with a physical NIC would likely be needed for zero-copy to surpass two-copy.

6.6 Unexpected result

The most unexpected result is the **non-monotonic latency behavior**. For 1K messages, two-copy latency jumps to $\sim$500 μs at 8 threads while zero-copy *drops* to $\sim$50 μs. For 64K, two-copy latency *decreases* from $\sim$275 μs (1 thread) to $\sim$110 μs (8 threads).

This is explained by: (1) per-message latency decreasing when throughput parallelism increases the total message count; (2) kernel TCP batching becoming more effective at higher thread counts; and (3) the i7-12700's P/E-core heterogeneity causing scheduling-dependent performance variations. The two-copy spike at 1K/8-threads specifically sug-

gests socket buffer lock contention for small, frequent sends—a cost zero-copy avoids by bypassing the kernel copy path.

7 AI Tool Usage Declaration

As required by the course policy, this section provides a detailed and explicit declaration of all components of this assignment where Large Language Model (LLM) tools were used, including the nature of usage and representative prompts. AI tools were extensively used throughout the assignment; however, all outputs were critically reviewed, modified, tested, and validated by me before final submission. I take full responsibility for the correctness of the code, results, and analysis.

7.1 Components Where AI Assistance Was Used

7.1.1 Code Development and Implementation

AI tools were used during the development of the C client–server programs to:

- Generate initial code skeletons for socket programming
- Suggest correct usage patterns for `send()`, `sendmsg()`, `iovec`, and `MSG_ZEROCOPY`
- Help refactor code to meet assignment constraints
- Debug compilation errors, runtime crashes, and incorrect behavior

Example Prompts:

> "Write a multithreaded TCP server in C with one thread per client."
>
> "How to use sendmsg() with iovec for scatter-gather I/O in Linux?"
>
> "Check whether this zero-copy socket code using MSG_ZEROCOPY is correct."

Validation: All AI-generated code was manually reviewed, modified, compiled, executed, and tested by me.

7.1.2 Experimental Design and Data Collection

AI tools assisted in selecting message sizes, thread counts, writing automation scripts, and understanding `perf stat` counters. All experiments were executed by me on my local machine and CSV files contain real measured data.

7.1.3 Plot Generation and Report Writing

AI tools helped generate matplotlib scripts and improve LaTeX formatting. Plots were generated from real experimental data. All final interpretations and conclusions are my own.

7.2 Responsibility Statement

- All code was compiled, executed, and debugged by me
- All experiments were run on the local testbed (Intel i7-12700)
- All results correspond to real measurements
- All final interpretations and conclusions are my own

8 Conclusion

This assignment demonstrates that data movement cost in network I/O is workload-dependent. On this testbed (veth pair, no real NIC), two-copy consistently achieved the highest throughput due to optimized `memcpy()` and cache-friendly contiguous buffers. Zero-copy incurred the highest per-byte CPU cost from page pinning overhead without real DMA benefit. Latency behavior was non-monotonic due to interactions between parallelism, kernel batching, and CPU core heterogeneity. Cache miss patterns were noisy, highlighting the difficulty of clean profiling on modern systems. Zero-copy techniques are most beneficial with real hardware DMA and large transfer sizes.

9 GitHub Repository

Repository URL:

https://github.com/ImPiyushK/GRS_PA02